GEMMAOS

PHASE 1 — DESKTOP FOUNDATION

OBJECTIVE

Build a modular, browser-accessible desktop environment that establishes the foundation for GemmaOS as a future bootable Linux operating system.

Phase 1 must prioritize modularity, separation of concerns, and platform independence. The architecture must support migration to Linux without requiring a rewrite of application logic or core business logic.

---

1. CORE SYSTEM COMPONENTS

Desktop Shell

- Desktop rendering and interaction.
- Desktop icons and navigation.
- Taskbar and application launcher.
- System tray.
- Desktop customization.

Window Manager

- Window creation and destruction.
- Window positioning and resizing.
- Window focus and stacking.
- Minimize, maximize and restore.
- Window lifecycle management.

Application Registry

- Centralized application registration.
- Application metadata.
- Application launching and termination.
- Application lifecycle management.
- Application permissions.

Application Framework

- Standardized application interfaces.
- Independent application execution contexts.
- Shared system service access.
- Application state management.

Core System Services

- Event Bus.
- State Manager.
- Settings Manager.
- Notification Service.
- Persistence Service.
- File Service.
- Permission Manager.
- Theme Engine.

---

2. TECHNOLOGY STACK

Proposed technologies:

- React
- TypeScript
- Vite
- Zustand
- Tailwind CSS
- Framer Motion
- IndexedDB

Use strict TypeScript typing and modular component architecture.

Avoid tightly coupling application logic to browser-specific APIs.

---

3. PLATFORM ABSTRACTION LAYER

All platform-dependent functionality must be accessed through interfaces.

Establish abstractions for:

- IFileSystem
- IStorageProvider
- INotificationService
- IModelRuntime
- IHardwareProvider
- IExecutionEnvironment

Phase 1 will implement browser-compatible adapters.

Phase 2 and subsequent Linux integration will implement native Linux adapters.

Applications and business logic must communicate through these abstractions rather than directly accessing platform-specific implementations.

---

4. INITIAL APPLICATION FRAMEWORK

Establish the following initial applications:

1. File Explorer
2. System Settings
3. Application Launcher
4. Developer Workspace
5. System Monitor

Applications must be independently registered and launched through the centralized Application Registry.

Gemma and EmbeddingGemma 2 integrations belong to Phase 2.

Do not prematurely implement planned AI infrastructure during Phase 1.

---

5. SYSTEM ARCHITECTURE

Organize the system into distinct layers:

1. Presentation Layer

    - Desktop Shell
    - Window Manager
    - UI Components

2. Application Layer
    - Application Registry
    - Application Framework
    - Individual Applications
3. Core Services Layer
    - Event Bus
    - State Management
    - Settings
    - Notifications
    - Permissions
4. Platform Abstraction Layer
    - Filesystem Interface
    - Storage Interface
    - Hardware Interface
    - Execution Interface
    - Notification Interface
5. Infrastructure Layer
    - Browser Adapters
    - IndexedDB
    - Browser APIs

Maintain strict dependency boundaries between layers.

---

6. VISUAL DESIGN SYSTEM

GemmaOS must follow its established visual identity:

- Saturated lavender, violet and purple.
- Bubblegum pink and hot pink accents.
- Warm cream backgrounds and panels.
- Thick deep-plum outlines.
- Rounded toy-like UI components.
- Chunky simplified icons.
- Flat 2D illustration style.
- Playful gradients and soft shadows.
- Stars, hearts and sparkles.

Avoid:

- Glossy 3D renders.
- Photorealistic imagery.
- Muted monochrome palettes.
- Sterile corporate dashboards.

The desktop must feel like a cohesive operating environment rather than a collection of unrelated web applications.

---

7. PERSISTENCE

Implement persistent storage for:

- Desktop configuration.
- Application settings.
- Window preferences.
- Theme preferences.
- Application state.
- User preferences.

Use IndexedDB where appropriate.

Define storage interfaces so the underlying implementation can later migrate to native Linux storage.

---

8. DEVELOPMENT MILESTONES

P1.1 — Repository Architecture and Project Foundation

P1.2 — Desktop Shell

P1.3 — Window Manager

P1.4 — Application Framework

P1.5 — Core System Services

P1.6 — Persistence Infrastructure

P1.7 — Theme Engine and Settings

P1.8 — Integration Testing and Phase 1 Release

Complete and validate each milestone before treating its functionality as implemented.

---

9. ACCEPTANCE CRITERIA

Phase 1 is complete when:

- The desktop loads reliably.
- Applications can be registered and launched.
- Multiple windows operate independently.
- Window focus and stacking behave correctly.
- Applications communicate through defined interfaces.
- Core services operate independently of individual applications.
- User settings persist across sessions.
- Application state can be recovered.
- Platform-dependent functionality is abstracted.
- The architecture is ready for subsequent AI integration.
- The architecture supports eventual Linux migration.

---

10. IMPLEMENTATION RULES

1. Inspect the existing repository before modifying files.
2. Preserve working functionality.
3. Do not introduce unnecessary dependencies.
4. Avoid monolithic components.
5. Maintain strict TypeScript contracts.
6. Document architectural decisions.
7. Test individual subsystems.
8. Never represent planned functionality as implemented.
9. Maintain clear boundaries between applications and system services.
10. Keep Phase 2 AI integration requirements in mind without prematurely implementing them.

FIRST TASK: P1.1

Begin by inspecting the existing repository.

Report:

- Existing project structure.
- Current technology stack.
- Existing implemented functionality.
- Architectural problems.
- Missing foundational components.
- Recommended directory structure.
- Dependency boundaries.
- Proposed implementation sequence.

Do not begin a large-scale rewrite.

Establish the architecture first, then implement incrementally.

LONG-TERM OBJECTIVE

GemmaOS will evolve through six phases:

1. Desktop Foundation.
2. AI Intelligence Infrastructure.
3. Agent Operating Environment.
4. System Integration and Hardening.
5. Linux Migration.
6. Bootable OS and Release.

All Phase 1 architectural decisions must preserve this trajectory.

GEMMAOS

PHASE 2 — AI INTELLIGENCE INFRASTRUCTURE

OBJECTIVE

Integrate generative and semantic intelligence into GemmaOS through independent model runtimes, a centralized AI orchestration interface, and shared semantic memory infrastructure.

Phase 2 establishes the AI infrastructure required for Phase 3's Agent Operating Environment.

GemmaOS must treat AI as a first-class system subsystem rather than an isolated chatbot application.

The architecture must support multiple model providers, independent model execution, resource-aware inference, and extensible semantic retrieval.

---

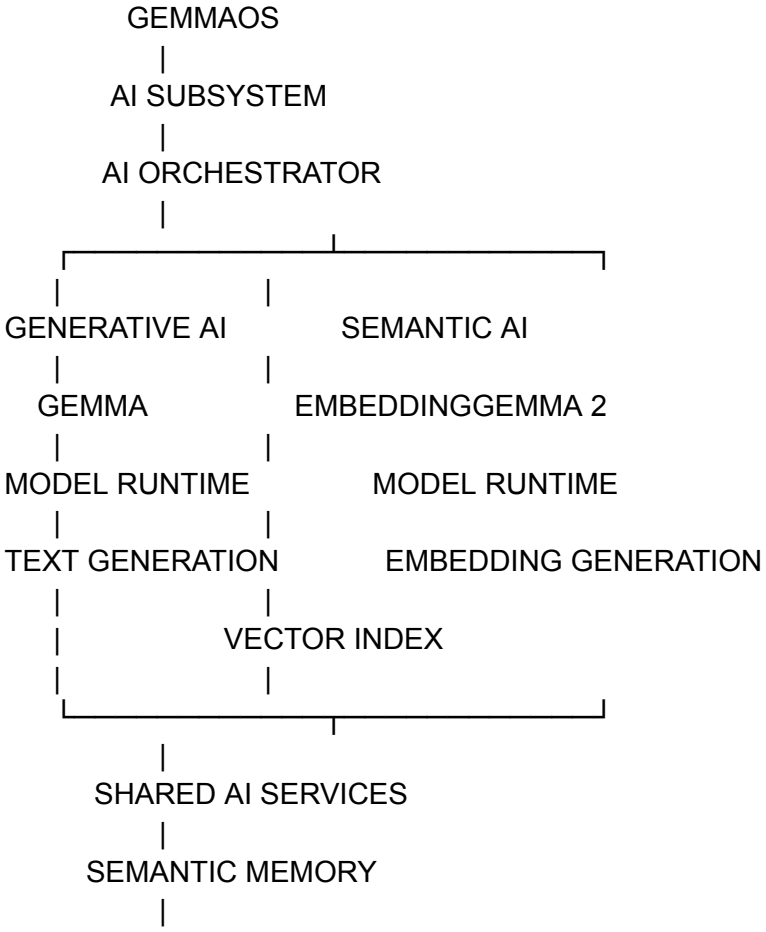
1. CORE ARCHITECTURE

The AI Intelligence Infrastructure consists of five primary components:

1. Generative Intelligence — Gemma.
2. Semantic Intelligence — EmbeddingGemma 2.
3. Model Runtime Manager.
4. AI Orchestration Interface.
5. Semantic Memory Infrastructure.

These components must remain independently modular.

SYSTEM ARCHITECTURE



CONTEXT RETRIEVAL
|
DATA STORAGE

---

2. GENERATIVE INTELLIGENCE — GEMMA

Gemma serves as the primary generative intelligence engine.

Its responsibilities include:

- Natural-language understanding.
- Text generation.
- Reasoning and planning.
- Conversational interaction.
- Structured output generation.
- Contextual response generation.
- Tool-calling support where available.
- Interaction with authorized system services.

Required infrastructure

Create a provider-independent generative model interface.

Example:

```
interface IGenerativeModel {
  initialize(): Promise<void>;

  generate(
    request: GenerationRequest
  ): AsyncIterable<GenerationEvent>;

  cancel(requestId: string): Promise<void>;

  getStatus(): Promise<ModelStatus>;

  shutdown(): Promise<void>;
}
```

The implementation must support:

- Streaming inference.
- Conversation context.
- System instructions.
- Generation configuration.
- Request cancellation.
- Error handling.
- Model health monitoring.

Do not hard-code Gemma directly into the desktop UI.

Applications must communicate with the model through the AI subsystem.

---

3. SEMANTIC INTELLIGENCE — EMBEDDINGGEMMA 2

EmbeddingGemma 2 serves as the semantic representation engine.

Unlike Gemma, it does not primarily generate conversational responses.

Its purpose is to transform supported input data into numerical representations that can be indexed and searched.

Responsibilities

- Text embeddings.
- Source-code embeddings.
- Supported multimodal embeddings.
- Semantic similarity.
- Document indexing.
- Memory indexing.
- Query embedding generation.
- Semantic retrieval.

Use the appropriate model configuration and encoder capabilities for the supported modalities.

Do not assume every model configuration supports every modality.

Embedding interface

interface IEmbeddingModel {

```
  initialize(): Promise<void>;

  embed(
    input: EmbeddingInput
  ): Promise<EmbeddingResult>;

  getDimensions(): number;

  getSupportedModalities(): Modality[];

  getStatus(): Promise<ModelStatus>;

  shutdown(): Promise<void>;
}
```

Embedding pipeline

```
     INPUT DATA
         |
   CONTENT EXTRACTION
         |
       CHUNKING
         |
   EMBEDDINGGEMMA 2
         |
   VECTOR GENERATION
         |
     VECTOR INDEX
         |
    SEMANTIC SEARCH
```

Embedding generation must be independent of generative inference.

---

4. MODEL RUNTIME MANAGER

Create a centralized Model Runtime Manager responsible for coordinating model execution.

Gemma and EmbeddingGemma 2 must have independent runtime instances.

Responsibilities

- Model initialization.
- Model loading and unloading.
- Runtime selection.
- Inference scheduling.
- Resource monitoring.
- Model configuration.
- Request management.
- Streaming support.
- Runtime health checks.
- Error recovery.
- Model lifecycle management.

Runtime abstraction

```
interface IModelRuntime {
  load(model: ModelConfiguration): Promise<void>;

  unload(modelId: string): Promise<void>;

  getStatus(modelId: string): Promise<ModelStatus>;

  getResourceUsage(): Promise<ResourceMetrics>;
}
```

Execution strategies

Support three execution modes:

1. Local inference.
2. Remote inference through authenticated providers.
3. Hybrid execution.

Phase 2 should prioritize reliable remote inference where local hardware is unavailable.

Local execution should remain an architectural capability rather than an assumed feature.

Never expose API keys or provider credentials to browser-side code.

---

5. AI ORCHESTRATION INTERFACE

Create a centralized AI orchestration service.

Its responsibility is to coordinate access to the available intelligence components.

It should not become a monolithic implementation of every AI feature.

Responsibilities

- Model selection.
- Request routing.
- Context construction.
- Generative inference coordination.
- Embedding request coordination.
- Retrieval integration.
- Response streaming.
- Request lifecycle management.

Example:

```
interface IAIOrchestrator {
  generate(
    request: AIRequest
  ): AsyncIterable<AIResponseEvent>;

  embed(
    request: EmbeddingInput
  ): Promise<EmbeddingResult>;

  retrieve(
    query: string,
    options?: RetrievalOptions
  ): Promise<RetrievalResult[]>;

  getSystemStatus(): Promise<AISystemStatus>;
}
```

The orchestration interface should provide a stable API for future applications and the Phase 3 agent subsystem.

Do not implement the full autonomous agent execution environment in Phase 2.

---

6. SEMANTIC MEMORY INFRASTRUCTURE

Build semantic memory as a shared system service.

It must be accessible to authorized applications and future agents.

Components

Content Ingestion

- Document ingestion.
- Text extraction.
- Code ingestion.
- Supported media ingestion.
- Metadata extraction.

Processing

- Content normalization.
- Chunking.
- Embedding generation.
- Metadata association.
- Index preparation.

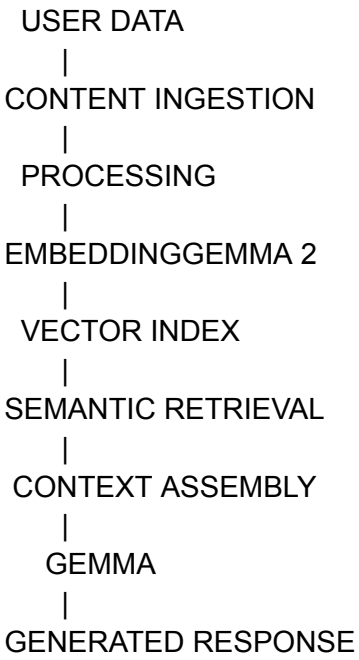
Retrieval

- Query embedding.
- Vector similarity search.
- Metadata filtering.
- Relevance ranking.
- Context assembly.

Persistence

- Original content references.
- Embedding vectors.
- Metadata.
- Index configuration.
- Retrieval history where appropriate.

Proposed data flow



Initial use cases

- Semantic file search.
- Conversation retrieval.
- Persistent knowledge retrieval.
- Developer workspace search.
- Codebase retrieval.
- Document discovery.
- Contextual application search.

The system must preserve original source references so retrieved information can be traced back to its source.

---

7. DATA STORAGE AND VECTOR INFRASTRUCTURE

Create storage abstractions for:

- Embedding vectors.
- Document metadata.

- Content references.
- Model configurations.
- Memory records.
- Index metadata.

Potential implementation technologies:

- SQLite for structured metadata.
- A vector database or vector-search extension for similarity indexing.
- IndexedDB for browser-compatible development.
- Filesystem-backed storage during Linux integration.

Select the actual storage implementation based on measured requirements.

Do not introduce multiple databases without a clear architectural reason.

Important requirements

- Versioned embedding records.
- Model and configuration identification.
- Index rebuilding.
- Duplicate detection.
- Deletion propagation.
- Metadata filtering.
- Data ownership.
- Backup and recovery.

Changing embedding models or dimensions must not silently corrupt existing indexes.

---

8. INITIAL AI APPLICATIONS

Introduce the following applications through the Phase 1 Application Framework.

AI Assistant

- Conversational Gemma interface.
- Streaming responses.
- Conversation history.
- Model configuration.

Semantic Search

- Search documents by meaning.
- Search indexed code.
- Retrieve relevant conversations.
- Display source references.

Memory Manager

- Browse indexed memories.
- Inspect memory metadata.
- Remove or re-index records.
- Monitor indexing status.

Model Manager

- View available models.
- Inspect runtime status.
- Configure providers.
- Monitor model resources.
- Load and unload models where supported.

These applications must consume shared AI services rather than implement independent model integrations.

---

9. SECURITY REQUIREMENTS

- Keep provider credentials server-side.
- Authenticate remote inference requests.
- Validate model inputs.
- Enforce data-access permissions.
- Isolate user data.
- Validate retrieved content.
- Prevent unauthorized filesystem access.
- Log important runtime events.
- Provide controlled model shutdown.
- Avoid exposing sensitive information in telemetry.

Semantic retrieval must respect the permissions of the original content.

---

10. DEVELOPMENT MILESTONES

P2.1 — AI Subsystem Foundation

Establish AI service architecture, interfaces, request contracts, and dependency boundaries.

P2.2 — Gemma Runtime

Implement generative inference, streaming, provider adapters, and model configuration.

P2.3 — EmbeddingGemma 2 Runtime

Implement embedding generation, supported modality handling, and independent model lifecycle management.

P2.4 — Model Runtime Manager

Implement model registration, loading, unloading, resource monitoring, and execution management.

P2.5 — Semantic Memory Infrastructure

Implement ingestion, chunking, embeddings, indexing, and retrieval.

P2.6 — AI Orchestration

Integrate model routing, context construction, retrieval, and generation.

P2.7 — AI Applications

Integrate AI Assistant, Semantic Search, Memory Manager, and Model Manager.

P2.8 — Integration Testing

Validate model execution, retrieval quality, persistence, security, and system reliability.

---

11. ACCEPTANCE CRITERIA

Phase 2 is complete when:

- Gemma inference works through a standardized runtime interface.
- Streaming generation works reliably.
- EmbeddingGemma 2 generates valid embeddings.
- Both models operate independently.
- The Model Runtime Manager monitors their lifecycle.
- Semantic indexing works.
- Semantic retrieval returns relevant source references.
- Retrieved context can be supplied to Gemma.
- Memory and embedding records persist.
- Model providers can be changed without rewriting applications.
- API credentials remain protected.
- Runtime errors are handled gracefully.
- The AI applications integrate with the existing desktop framework.

---

12. ARCHITECTURAL BOUNDARIES

Phase 2 establishes intelligence infrastructure.

It does NOT attempt to implement the complete agent operating environment.

The following belong primarily to Phase 3:

- Autonomous agent lifecycle management.
- Multi-agent orchestration.
- Complex task decomposition.
- Workflow execution.
- Advanced tool permissions.
- Agent-to-agent communication.
- Background agent execution.

Phase 2 must expose stable interfaces that allow Phase 3 to build on its infrastructure.

FINAL ARCHITECTURAL OBJECTIVE

GemmaOS must contain two independent model systems:

GENERATIVE INTELLIGENCE:
Gemma handles language generation, reasoning, and contextual interaction.

SEMANTIC INTELLIGENCE:
EmbeddingGemma 2 handles semantic representation, indexing, and retrieval.

The AI Orchestrator coordinates access to both.

The Semantic Memory Infrastructure provides shared retrieval capabilities across the operating environment.

These components must remain independently replaceable, observable, and extensible.

The result is an AI-native computing architecture that can eventually migrate from the browser-based Phase 1 environment into native Linux services without rewriting its core intelligence interfaces.

GEMMAOS

PHASE 3 — AGENT OPERATING ENVIRONMENT

OBJECTIVE

Develop an integrated agent operating environment that enables AI agents to interact with GemmaOS applications, access authorized system services, execute tools, coordinate workflows, and perform multi-step tasks.

Phase 3 builds on the intelligence infrastructure established in Phase 2.

Gemma provides generative intelligence.

EmbeddingGemma 2 provides semantic representation and retrieval.

The Agent Operating Environment provides the infrastructure through which agents can use these capabilities to perform meaningful work.

Agents must operate through controlled system interfaces rather than receiving unrestricted access to the operating environment.

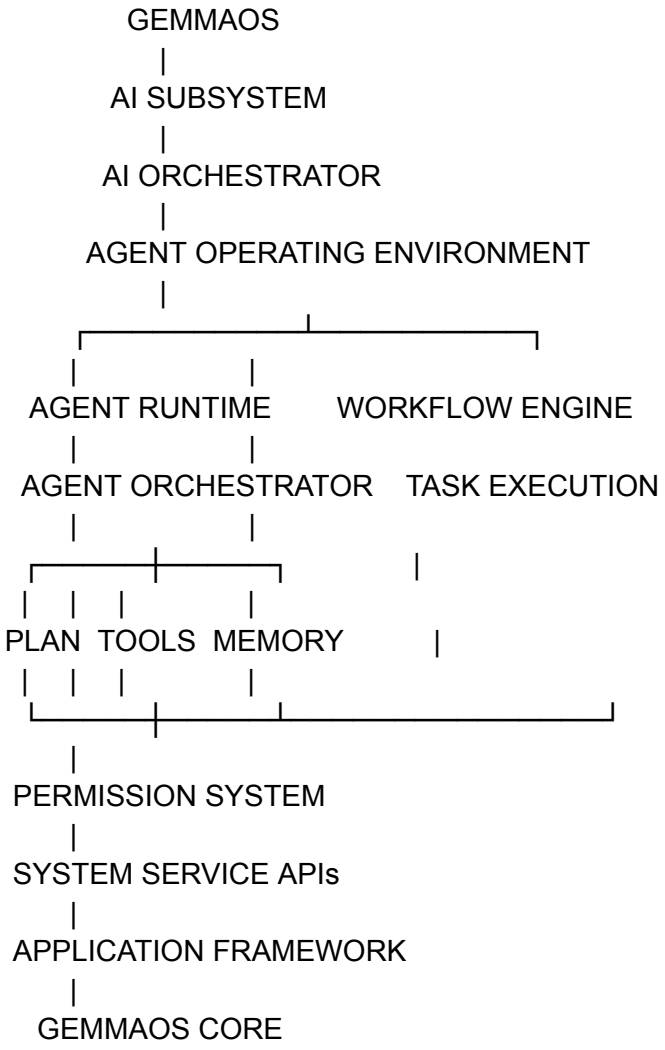
---

1. CORE ARCHITECTURE

The Agent Operating Environment consists of eight primary subsystems:

1. Agent Runtime.
2. Agent Orchestrator.
3. Task Planning and Execution.
4. Tool Registry.
5. Permission and Authorization System.
6. Workflow Engine.
7. Agent Memory Integration.
8. Agent Observability.

SYSTEM ARCHITECTURE



The Agent Operating Environment must communicate with the existing AI subsystem and Core Services through documented interfaces.

---

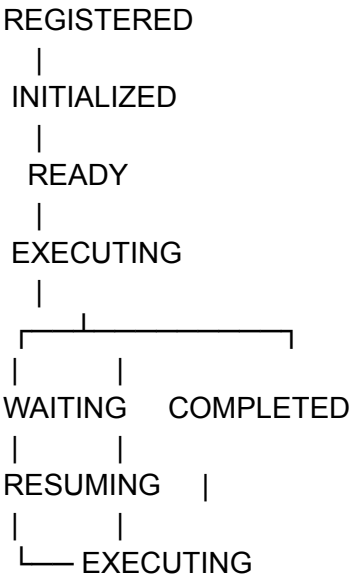
2. AGENT RUNTIME

Create an independent runtime responsible for agent lifecycle management.

Responsibilities

- Agent initialization.
- Agent registration.
- Agent execution.
- Agent suspension.
- Agent termination.
- Agent configuration.
- Agent state management.
- Execution context management.
- Resource limits.
- Error recovery.

Agent lifecycle



Additional terminal states:

- Failed.
- Cancelled.
- Suspended.
- Awaiting approval.

Agents must have explicit lifecycle states.

The runtime must support cancellation and controlled shutdown.

---

3. AGENT ORCHESTRATOR

The Agent Orchestrator coordinates agent activity.

It is responsible for managing requests between agents, models, tools, and system services.

Responsibilities

- Agent selection.
- Task routing.
- Execution coordination.
- Context construction.
- Task delegation.
- Result aggregation.
- Execution monitoring.
- Failure handling.
- Retry policies.
- Agent communication.

The orchestrator should remain separate from the individual agent implementations.

Architectural requirement

The orchestration system must support different generative model providers.

Do not hard-code the orchestration layer to Gemma alone.

---

4. TASK PLANNING AND EXECUTION

Create a task execution subsystem that allows agents to perform multi-step operations.

Responsibilities

- Task decomposition.

- Step generation.
- Dependency resolution.
- Execution ordering.
- Progress tracking.
- Result validation.
- Failure recovery.
- Task cancellation.

Execution pipeline

```
USER REQUEST
     |
GEMMA REASONING
     |
TASK DECOMPOSITION
     |
EXECUTION PLAN
     |
DEPENDENCY CHECK
     |
PERMISSION CHECK
     |
TOOL EXECUTION
     |
RESULT VALIDATION
     |
NEXT EXECUTION STEP
     |
COMPLETION
```

Task plans must be inspectable.

Agents should expose what they intend to do before executing consequential actions.

Support human approval for operations that modify important user data, access sensitive resources, or perform external actions.

---

5. TOOL REGISTRY

Create a centralized registry of tools available to agents.

Tools are standardized interfaces through which agents interact with GemmaOS.

Initial tool categories

Filesystem Tools

- Read files.
- Create files.
- Modify files.
- Search directories.
- Retrieve metadata.

Application Tools

- Launch applications.
- Inspect application state.
- Communicate with authorized applications.
- Manage application sessions.

Memory Tools

- Search semantic memory.
- Retrieve documents.
- Store approved memories.
- Inspect retrieval results.

System Tools

- Inspect system status.
- Retrieve resource metrics.
- Read system configuration.
- Access authorized system services.

Developer Tools

- Inspect project files.
- Search source code.
- Run approved tests.
- Retrieve build information.

Tool execution must occur through controlled adapters.

Do not expose unrestricted operating system command execution to agents.

---

6. PERMISSION AND AUTHORIZATION SYSTEM

Security must be foundational to agent execution.

Create a centralized authorization subsystem.

Permission categories

- Read.
- Write.
- Execute.
- Delete.
- Network access.
- Application access.
- Memory access.
- System configuration access.

Permission model

Every tool must declare:

- Required permissions.
- Accessible resources.
- Expected side effects.
- Execution constraints.
- Whether human approval is required.

Execution policies

Classify operations into three categories:

Category A — Read-only

Examples:

- Search documents.
- Inspect application state.

- Retrieve system information.

May execute automatically when the agent has the necessary access.

Category B — Reversible modifications

Examples:

- Create a draft.
- Modify a temporary document.
- Change a reversible application setting.

Require appropriate authorization and maintain recovery options.

Category C — Sensitive or consequential actions

Examples:

- Delete important files.
- Modify system configuration.
- Execute external transactions.
- Transmit private information.

Require explicit approval and additional validation.

The authorization system must enforce permissions independently of model-generated instructions.

Never treat a model's assertion that an operation is authorized as authorization.

---

7. WORKFLOW ENGINE

Create a workflow engine for coordinating multi-step operations.

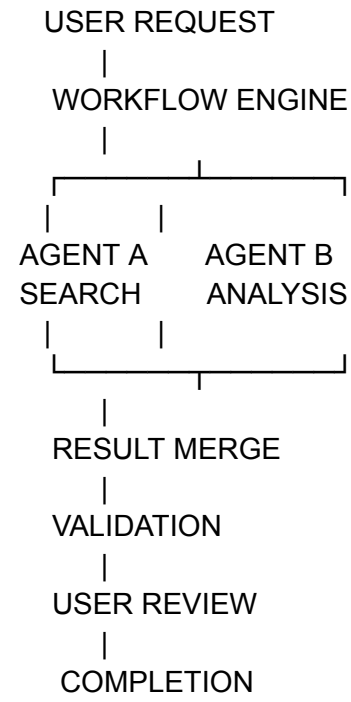
Workflows should be independent of individual agents.

Capabilities

- Sequential execution.
- Conditional branching.
- Parallel tasks.
- Task dependencies.

- Retry policies.
- Timeouts.
- Execution checkpoints.
- Cancellation.
- Recovery.

Example workflow



Workflows must preserve execution state so interrupted operations can resume safely.

---

8. AGENT MEMORY INTEGRATION

Connect the Agent Operating Environment to Phase 2's Semantic Memory Infrastructure.

Agents should have access to relevant contextual information without receiving the entire memory database.

Capabilities

- Contextual memory retrieval.
- Conversation history retrieval.

- Semantic document search.
- Task-specific context.
- Persistent agent state.
- Approved memory creation.
- Memory provenance.

Use EmbeddingGemma 2 for semantic retrieval.

Use Gemma or another configured generative model for planning and generation.

Memory boundaries

Support:

- Shared system memory.
- User-owned memory.
- Agent-specific memory.
- Task-specific temporary context.

Enforce ownership and access permissions.

Private agent memory must not automatically become globally accessible.

---

9. MULTI-AGENT COORDINATION

Establish the architecture for multiple agents working together.

Required capabilities

- Agent registration.
- Agent identification.
- Agent-to-agent messaging.
- Task delegation.
- Shared task context.
- Result aggregation.
- Conflict resolution.
- Execution coordination.

Each agent should have:

- Unique identifier.

- Configuration.
- Model assignment.
- Available tools.
- Permission profile.
- Execution state.
- Memory scope.

Important architectural rule

Agents should communicate through the orchestration infrastructure rather than directly modifying each other's internal state.

The system must support both single-agent and multi-agent execution.

---

10. AGENT OBSERVABILITY

Create a centralized monitoring interface.

Expose:

- Active agents.
- Agent lifecycle states.
- Current tasks.
- Tool execution history.
- Permission requests.
- Approval status.
- Model requests.
- Execution latency.
- Errors.
- Resource consumption.

Maintain an auditable record of consequential agent actions.

Telemetry must reflect actual runtime measurements rather than simulated values.

---

11. INITIAL AGENT APPLICATIONS

Integrate the following applications into GemmaOS.

Agent Manager

- Register agents.
- Configure agents.
- Inspect agent status.
- Start and stop agents.
- Assign permissions.

Workflow Manager

- Create workflows.
- Inspect execution plans.
- Monitor progress.
- Resume interrupted workflows.
- Review completed workflows.

Tool Manager

- Browse available tools.
- Inspect tool capabilities.
- Configure permissions.
- Monitor tool execution.

Agent Activity Monitor

- View active executions.
- Inspect task history.
- Review errors.
- Inspect approval requests.

These applications must use shared Agent Operating Environment services.

---

12. DEVELOPMENT MILESTONES

P3.1 — Agent Runtime Foundation

Establish agent interfaces, lifecycle management, registration, and execution contexts.

P3.2 — Agent Orchestrator

Implement agent routing, coordination, context management, and execution control.

P3.3 — Task Planning and Execution

Implement task decomposition, execution plans, dependencies, validation, and cancellation.

P3.4 — Tool Registry

Implement standardized tool registration, discovery, execution, and result handling.

P3.5 — Permission and Authorization System

Implement permission policies, resource boundaries, approval workflows, and audit records.

P3.6 — Workflow Engine

Implement sequential execution, branching, parallel operations, checkpoints, and recovery.

P3.7 — Agent Memory Integration

Connect agent contexts to semantic retrieval and scoped persistent memory.

P3.8 — Multi-Agent Coordination

Implement messaging, delegation, shared task context, and result aggregation.

P3.9 — Observability and Agent Applications

Build management interfaces and execution monitoring.

P3.10 — Integration Testing

Validate agent execution, authorization, recovery, and coordination.

---

13. ACCEPTANCE CRITERIA

Phase 3 is complete when:

- Agents can be registered and initialized.
- Agent lifecycle states are managed reliably.
- Agents can receive and execute tasks.
- Multi-step task plans can be created and inspected.
- Agents can invoke registered tools.

- Tool permissions are independently enforced.
- Sensitive operations require appropriate approval.
- Workflows support dependencies and recovery.
- Agents can retrieve authorized semantic memory.
- Multiple agents can coordinate through the orchestrator.
- Agent activity is observable and auditable.
- Failed and interrupted tasks can be handled safely.
- Agent services integrate with the Phase 1 and Phase 2 architecture.

---

14. ARCHITECTURAL BOUNDARIES

Phase 3 builds the agent execution environment.

It does not replace:

- Phase 1 Desktop Foundation.
- Phase 2 AI Intelligence Infrastructure.

It consumes their established interfaces.

It must also prepare for Phase 4's security, persistence, resource management, and reliability work.

FINAL ARCHITECTURAL OBJECTIVE

GemmaOS must support an extensible environment in which multiple AI agents can:

- Understand user requests.
- Plan tasks.
- Retrieve relevant information.
- Access authorized tools.
- Interact with applications.
- Coordinate with other agents.
- Execute workflows.
- Preserve task state.
- Report their actions.

The Agent Operating Environment transforms GemmaOS from an AI-enabled desktop into an extensible agent-native computing environment.

All agent capabilities must remain subject to explicit system permissions and controlled execution boundaries.

GEMMAOS

PHASE 4 — SYSTEM INTEGRATION & HARDENING

OBJECTIVE

Harden and integrate the subsystems developed during Phases 1–3.

Phase 4 focuses on system-wide reliability, security, persistence, resource management, observability, and testing.

The objective is to establish a stable, maintainable software platform that is ready for migration into a native Linux environment during Phase 5.

This phase must address architectural weaknesses before introducing hardware-dependent operating system infrastructure.

---

1. SYSTEM INTEGRATION

Integrate all existing GemmaOS subsystems into a cohesive architecture.

Components

- Desktop Shell.
- Window Manager.
- Application Framework.
- Core System Services.
- Gemma Runtime.
- EmbeddingGemma 2 Runtime.
- Model Runtime Manager.
- AI Orchestrator.
- Semantic Memory Infrastructure.
- Agent Runtime.

- Agent Orchestrator.
- Workflow Engine.
- Tool Registry.
- Permission System.

Responsibilities

- Validate service communication.
- Standardize internal APIs.
- Resolve dependency conflicts.
- Validate event propagation.
- Establish service lifecycle dependencies.
- Verify application-to-service communication.
- Eliminate duplicated functionality.
- Identify architectural bottlenecks.

Acceptance criteria

All major subsystems must communicate through documented interfaces.

No subsystem should require direct access to another subsystem’s private implementation details.

---

2. PERSISTENCE & DATA INTEGRITY

Establish reliable persistence across the entire operating environment.

Data categories

- User preferences.
- Desktop configuration.
- Application state.
- Window state.
- Conversations.
- Agent configurations.
- Agent execution history.
- Workflow checkpoints.
- Semantic memories.
- Embedding records.
- Model configurations.
- System settings.

Required capabilities

- Persistent storage.
- Transaction-safe updates.
- Data validation.
- Schema versioning.
- Migration support.
- Backup and restoration.
- Recovery after interrupted operations.
- Duplicate detection.
- Data integrity verification.

Architectural requirements

Separate transient runtime state from persistent application and system data.

Define explicit ownership for every persistent data category.

Ensure embedding indexes can be reconstructed from their original source records.

Deleting source content must propagate to associated embeddings and indexes.

---

3. SECURITY ARCHITECTURE

Implement system-wide security boundaries.

Authentication

- User identity.
- Session management.
- Authentication state.
- Credential handling.

Authorization

- Application permissions.
- Agent permissions.
- Filesystem access.
- Tool execution permissions.
- System configuration permissions.
- Network access policies.

Secrets management

- Secure API credentials.
- Environment-specific configuration.
- Server-side secret storage.
- Credential rotation support.
- No client-side exposure of private credentials.

Agent security

- Sandboxed execution where supported.
- Explicit tool permissions.
- Resource restrictions.
- Human approval for consequential operations.
- Execution auditing.
- Protection against unauthorized privilege escalation.

Data security

- Access-controlled memory retrieval.
- User-data isolation.
- Protected sensitive records.
- Secure communication with external providers.
- Controlled data export.

Threat modeling

Document:

- Trust boundaries.
- Attack surfaces.
- Privilege boundaries.
- External dependencies.
- Potential abuse scenarios.
- Mitigation strategies.

Never rely on model-generated instructions as a security enforcement mechanism.

Security decisions must be enforced by trusted system components.

---

4. RESOURCE MANAGEMENT

Establish system-wide resource monitoring and allocation.

Model resources

- Model memory consumption.
- Model initialization time.
- Inference latency.
- Concurrent inference requests.
- Model loading and unloading.
- Embedding throughput.

Agent resources

- Active agent count.
- Concurrent executions.
- Task duration.
- Tool execution limits.
- Background processing.
- Workflow resource consumption.

System resources

- CPU utilization.
- Memory consumption.
- Storage utilization.
- Network activity.
- Application performance.

Resource management capabilities

- Execution limits.
- Request queues.
- Concurrency control.
- Background task scheduling.
- Cancellation.
- Timeout handling.
- Resource-aware model selection.

Resource telemetry must reflect measured values.

Do not assume local GPU availability.

---

5. OBSERVABILITY & DIAGNOSTICS

Build a centralized observability infrastructure.

System monitoring

Expose:

- Application health.
- Service availability.
- Runtime status.
- Storage health.
- System errors.
- Application performance.

AI monitoring

Expose:

- Active model.
- Model provider.
- Inference latency.
- Token throughput where available.
- Embedding generation latency.
- Retrieval performance.
- Model loading status.

Agent monitoring

Expose:

- Active agents.
- Current tasks.
- Workflow execution state.
- Tool execution history.
- Permission requests.
- Failed operations.

Logging

Implement structured logs with:

- Timestamp.
- Subsystem identifier.

- Event category.
- Severity.
- Correlation ID.
- Execution identifier where applicable.

Sensitive information must not be written into logs unnecessarily.

---

6. ERROR HANDLING & RECOVERY

Establish consistent failure handling across GemmaOS.

Required capabilities

- Centralized error classification.
- Structured error reporting.
- Retry policies.
- Timeout handling.
- Graceful service degradation.
- Request cancellation.
- Interrupted workflow recovery.
- Application crash recovery.
- Model failure recovery.

Recovery scenarios

1. Generative model becomes unavailable.
2. Embedding model fails during indexing.
3. Vector index becomes corrupted.
4. Agent execution is interrupted.
5. Application crashes.
6. Persistent storage becomes unavailable.
7. External provider times out.
8. Workflow fails midway through execution.

System failures must not silently corrupt persistent state.

Operations with external side effects must support appropriate idempotency and recovery mechanisms.

---

7. TESTING INFRASTRUCTURE

Establish comprehensive automated testing.

Unit testing

Test individual services, interfaces, and business logic.

Integration testing

Test communication between:

- Desktop and application framework.
- Applications and core services.
- Gemma and AI orchestration.
- EmbeddingGemma 2 and semantic retrieval.
- Agents and tool registry.
- Agents and permission services.
- Workflow engine and persistence.

End-to-end testing

Validate complete user workflows.

Security testing

Test:

- Unauthorized access.
- Permission bypass attempts.
- Credential exposure.
- Unsafe tool execution.
- Memory access isolation.
- Invalid external inputs.

Performance testing

Measure:

- Application startup.
- Window interaction.
- Model initialization.
- Inference latency.
- Retrieval latency.

- Indexing throughput.
- Concurrent task execution.

Regression testing

Ensure new changes do not break previously validated functionality.

---

8. PLATFORM PORTABILITY VALIDATION

Prepare the architecture for Phase 5 Linux migration.

Review all platform-dependent functionality.

Validate abstractions for:

- IFileSystem.
- IStorageProvider.
- INotificationService.
- IModelRuntime.
- IHardwareProvider.
- IExecutionEnvironment.

Requirements

- No unnecessary direct browser API dependencies in business logic.
- No browser-only assumptions in core services.
- Explicit platform adapters.
- Portable application contracts.
- Documented service dependencies.
- Replaceable storage implementations.
- Replaceable model execution providers.

Identify browser-specific functionality that requires a native Linux implementation.

Do not begin a full Linux migration during Phase 4.

---

9. DOCUMENTATION & ARCHITECTURAL GOVERNANCE

Maintain authoritative technical documentation.

Required documents:

- Master Architecture Specification.
- System Dependency Map.
- Service Interface Documentation.
- Data Ownership Specification.
- Security and Threat Model.
- AI Runtime Specification.
- Agent Execution Specification.
- Persistence and Recovery Specification.
- Testing Strategy.
- Linux Migration Readiness Report.

All architecture documentation must distinguish:

IMPLEMENTED — Functionality exists and has been validated.

IN DEVELOPMENT — Implementation is incomplete.

PLANNED — Architectural intention only.

Documentation must be updated alongside significant architectural changes.

---

10. DEVELOPMENT MILESTONES

P4.1 — System-Wide Integration

Integrate and validate communication between all existing subsystems.

P4.2 — Persistence Hardening

Implement data integrity, migrations, backups, and recovery.

P4.3 — Security Hardening

Implement authentication, authorization, secret management, and threat mitigation.

P4.4 — Resource Management

Implement resource monitoring, concurrency limits, and scheduling.

P4.5 — Observability

Implement structured logging, diagnostics, and runtime monitoring.

P4.6 — Error Handling and Recovery

Implement consistent failure handling and recovery mechanisms.

P4.7 — Testing Infrastructure

Establish unit, integration, end-to-end, security, and performance testing.

P4.8 — Portability Validation

Audit platform dependencies and validate migration interfaces.

P4.9 — Documentation and Release Readiness

Complete architecture documentation and Phase 5 readiness assessment.

---

11. ACCEPTANCE CRITERIA

Phase 4 is complete when:

- All major subsystems integrate reliably.
- Service contracts are documented and validated.
- Persistent data survives application restarts.
- Interrupted workflows recover safely.
- Data integrity is verifiable.
- Security boundaries are enforced.
- Credentials remain protected.
- Agent permissions cannot be bypassed through model instructions.
- Resource usage is observable.
- Model failures are handled gracefully.
- System errors are logged consistently.

- Automated tests cover critical functionality.
- Platform-specific dependencies are identified.
- Linux migration requirements are documented.

---

12. ARCHITECTURAL BOUNDARIES

Phase 4 focuses on hardening the existing software platform.

It does not implement:

- Linux kernel integration.
- Native Linux desktop services.
- Bootloader configuration.
- Hardware driver integration.
- Final bootable installation images.

Those responsibilities belong to Phases 5 and 6.

Phase 4 must prepare the software architecture for those transitions without prematurely introducing platform-specific dependencies.

FINAL ARCHITECTURAL OBJECTIVE

GemmaOS must become a stable, secure, observable, and maintainable software platform.

Every major subsystem must have:

- Defined responsibilities.
- Explicit interfaces.
- Documented dependencies.
- Validated permissions.
- Reliable persistence.
- Failure handling.
- Automated tests.
- Observable runtime behavior.

The result should be a hardened AI-native computing environment ready to transition from browser-based development into a native Linux operating system.

GEMMAOS

PHASE 5 — LINUX MIGRATION

OBJECTIVE

Transition GemmaOS from its browser-based development environment into a native Linux desktop environment.

Phase 5 introduces native operating-system integration while preserving the modular architecture established during Phases 1–4.

The objective is to replace browser-specific infrastructure with native Linux implementations without rewriting application logic or core business services.

GemmaOS must become a native Linux application environment capable of interacting with the host operating system through controlled platform services.

This phase prepares the system for bootable distribution development in Phase 6.

---

1. LINUX DEVELOPMENT ENVIRONMENT

Establish a reproducible Linux development and testing environment.

Initial environment

Use a Linux virtual machine or cloud-hosted Linux environment during initial development.

Physical hardware is not required to begin this phase.

Responsibilities

- Establish a supported Linux distribution baseline.
- Configure the development toolchain.
- Define dependency management.
- Establish build and packaging workflows.
- Configure automated testing on Linux.
- Document runtime dependencies.
- Establish reproducible development environments.

Recommended approach

Start with a mainstream Linux distribution such as Ubuntu or Debian.

Keep distribution-specific functionality behind platform adapters wherever possible.

Do not make the entire application dependent on one distribution's package manager or desktop configuration.

---

2. NATIVE DESKTOP ENVIRONMENT

Transition the GemmaOS desktop shell into a native Linux desktop application.

Responsibilities

- Native application startup.
- Desktop shell rendering.
- Window management integration.
- Native keyboard and pointer input.
- Display configuration.
- Multi-window support.
- Application lifecycle management.
- Native notifications.
- System-level application launching.

Architecture

Preserve the existing desktop application architecture.

Introduce a native execution environment that hosts the desktop shell.

Evaluate an appropriate desktop hosting strategy, such as:

- Electron.
- Tauri.
- A native Linux application shell.

Select the approach based on:

- Existing frontend compatibility.
- Resource consumption.
- Native API access.

- Security boundaries.
- Packaging requirements.
- Long-term maintainability.

Do not introduce multiple desktop hosting frameworks unnecessarily.

Acceptance criteria

The GemmaOS desktop launches as a native Linux application rather than requiring a browser tab.

Existing desktop functionality remains operational.

---

3. NATIVE WINDOWING & DISPLAY INTEGRATION

Integrate with Linux display and windowing infrastructure.

Wayland integration

Target Wayland as the primary modern Linux display protocol.

Investigate the relationship between:

- Wayland compositor.
- Desktop shell.
- Window management.
- Application surfaces.
- Input devices.
- Display configuration.

Important architectural distinction

A desktop application running under Wayland is not automatically a complete Wayland compositor or independent desktop environment.

Initially, GemmaOS can operate as a native desktop shell hosted within an existing Linux desktop session.

If a fully independent desktop environment is pursued later, compositor development or integration must be treated as a separate engineering responsibility.

Required capabilities

- Window creation.
- Window focus.
- Window positioning where supported.
- Display scaling.
- Resolution awareness.
- Keyboard input.
- Pointer input.
- Multi-monitor compatibility.
- Application activation.

Acceptance criteria

The native GemmaOS desktop behaves reliably under a supported Wayland session.

---

4. NATIVE PLATFORM SERVICES

Replace browser-specific platform adapters with Linux implementations.

Preserve the existing platform abstraction interfaces.

Filesystem

Implement:

`ILinuxFileSystem`

Responsibilities:

- File creation.
- File reading and writing.
- Directory management.
- File metadata.
- File search.
- File permissions.
- File operations.

Filesystem access must respect user permissions and application authorization.

Storage

Implement:

`ILinuxStorageProvider`

Responsibilities:

- Persistent application data.
- System configuration.
- Local databases.
- Semantic index storage.
- Backup and restoration.
- Data migration.

Notifications

Implement:

`ILinuxNotificationService`

Integrate with native desktop notification infrastructure.

Hardware

Implement:

`ILinuxHardwareProvider`

Expose authorized system information and hardware capabilities.

Potential information includes:

- CPU.
- Memory.
- Storage.
- GPU availability.
- Battery status.
- Network interfaces.
- Connected peripherals.

Only report telemetry that can actually be measured.

Execution environment

Implement:

`ILinuxExecutionEnvironment`

Responsibilities:

- Native application lifecycle.
- Process management.
- Environment configuration.
- Runtime information.
- Controlled process execution.
- System service communication.

Acceptance criteria

Applications and core services interact with Linux through platform interfaces rather than directly accessing operating-system APIs.

---

5. AI RUNTIME MIGRATION

Integrate Gemma and EmbeddingGemma 2 into the native Linux environment.

The two models remain independent intelligence components.

The AI orchestrator continues to coordinate their capabilities.

Gemma runtime

Support:

- Remote inference.
- Local inference where hardware permits.
- Streaming generation.
- Model configuration.
- Runtime status.
- Model lifecycle.
- Cancellation.
- Error handling.

EmbeddingGemma 2 runtime

Support:

- Embedding generation.
- Semantic indexing.
- Similarity search.

- Document ingestion.
- Memory retrieval.
- Index persistence.
- Model configuration and versioning.

Model Runtime Manager

Extend the runtime manager to support native Linux execution providers.

Potential runtime backends should be evaluated according to model compatibility, hardware support, performance, and licensing.

Possible candidates include:

- llama.cpp.
- Ollama.
- Transformers-based execution.
- Authenticated remote inference providers.

Do not assume every backend supports every Gemma model configuration.

Hardware-aware execution

The system must detect available hardware and select appropriate execution strategies.

Support:

- CPU inference.
- GPU acceleration when available.
- Remote inference.
- Hybrid execution.

Local model execution must remain optional.

GemmaOS should still function without a dedicated GPU.

Acceptance criteria

The AI subsystem operates through the same documented orchestration interfaces regardless of whether inference is local or remote.

---

6. NATIVE APPLICATION EXECUTION

Transition the application framework into a native Linux application environment.

Responsibilities

- Application registration.
- Application launching.
- Application termination.
- Application lifecycle management.
- Window association.
- Application state persistence.
- Application permissions.
- Native application integration.

Application packaging

Define an application packaging strategy.

Evaluate:

- Native application bundles.
- Distribution packages.
- Flatpak compatibility.
- AppImage compatibility.

Avoid coupling application logic to a single packaging format.

Acceptance criteria

GemmaOS applications can launch and operate inside the native desktop environment.

---

7. SYSTEM SERVICE ARCHITECTURE

Introduce native Linux service integration.

Potential services

- Model Runtime Service.
- AI Orchestration Service.
- Semantic Memory Service.
- Agent Execution Service.
- Notification Service.

- System Monitoring Service.
- Persistence Service.

Responsibilities

- Service initialization.
- Service health monitoring.
- Dependency management.
- Graceful shutdown.
- Restart behavior.
- Inter-process communication.
- Configuration management.

Service management

Evaluate systemd integration for Linux service lifecycle management.

Keep service contracts independent of systemd so alternative environments remain possible.

Acceptance criteria

Critical background services can operate independently of the desktop application's visible interface.

---

8. NATIVE SECURITY & PERMISSIONS

Adapt the Phase 4 security architecture to Linux.

Required capabilities

- User-level permission enforcement.
- Filesystem access controls.
- Process isolation.
- Application sandboxing where appropriate.
- Secure credential storage.
- Network access controls.
- Agent execution restrictions.
- Auditable system operations.

Security considerations

Evaluate:

- Linux user and group permissions.
- Unix file permissions.
- Linux capabilities.
- D-Bus access controls.
- Application sandboxing.
- Secret-service integration.
- Process isolation.

Do not run the entire GemmaOS application as root.

Agents must not receive unrestricted host-system execution privileges.

Model-generated instructions must never override operating-system permissions or GemmaOS authorization policies.

Acceptance criteria

Native system access is mediated by explicit authorization boundaries.

---

9. HARDWARE ABSTRACTION

Prepare GemmaOS for operation across different computers.

Hardware discovery

Identify available:

- CPU architecture.
- RAM.
- Storage.
- GPU.
- Network adapters.
- Audio devices.
- Display devices.
- Input devices.

Hardware capability registry

Create a capability-oriented hardware abstraction.

Applications should request capabilities rather than assume particular devices exist.

Resource-aware behavior

- Select compatible inference backends.
- Avoid loading models beyond available memory.
- Monitor resource consumption.
- Support remote inference when local resources are insufficient.
- Degrade gracefully when optional hardware is unavailable.

Acceptance criteria

GemmaOS can adapt its execution strategy to different hardware configurations.

---

10. MIGRATION & COMPATIBILITY

Preserve existing application and user data.

Migration responsibilities

- Existing configuration migration.
- User preference migration.
- Application state migration.
- Conversation migration.
- Agent configuration migration.
- Workflow history migration.
- Semantic memory migration.
- Embedding index reconstruction where necessary.

Compatibility requirements

- Versioned migration procedures.
- Backup before migration.
- Data integrity validation.
- Rollback strategy.
- Schema compatibility.
- Migration logging.

Do not assume browser storage can be copied directly into native storage without transformation.

Acceptance criteria

Existing user data can be migrated into the native environment without silent loss or corruption.

---

11. DEVELOPMENT MILESTONES

P5.1 — Linux Development Foundation

Establish the Linux development environment and reproducible build pipeline.

P5.2 — Native Desktop Hosting

Host the GemmaOS desktop shell as a native Linux application.

P5.3 — Wayland Integration

Validate windowing, display, and input integration.

P5.4 — Native Platform Adapters

Implement Linux filesystem, storage, notification, hardware, and execution adapters.

P5.5 — Native AI Runtime

Integrate local and remote model execution providers.

P5.6 — Native Application Framework

Validate application registration, launching, lifecycle, and packaging.

P5.7 — Linux System Services

Introduce native background services and service management.

P5.8 — Security Integration

Implement Linux-specific security and authorization boundaries.

P5.9 — Hardware Abstraction

Implement hardware discovery and resource-aware execution.

P5.10 — Data Migration and Compatibility

Migrate existing data and validate persistence.

P5.11 — Native Integration Testing

Run system-wide compatibility, performance, security, and recovery tests.

P5.12 — Phase 6 Readiness Assessment

Produce a native Linux readiness report.

---

12. ACCEPTANCE CRITERIA

Phase 5 is complete when:

- GemmaOS launches as a native Linux desktop application.
- The desktop shell operates outside a browser tab.
- Wayland compatibility is validated.
- Native platform adapters are implemented.
- Existing applications operate in the native environment.
- Gemma and EmbeddingGemma 2 remain independently configurable.
- Local and remote inference are supported where compatible.
- Background system services operate reliably.
- Native permissions and security boundaries are enforced.
- Hardware capabilities can be discovered.
- Resource-aware execution works.
- User data can be migrated safely.
- Automated Linux integration tests pass.
- The native environment is documented and reproducible.

---

13. ARCHITECTURAL BOUNDARIES

Phase 5 establishes the native Linux environment.

It does not yet constitute the final bootable operating-system release.

Responsibilities reserved for Phase 6 include:

- Bootloader integration.
- Bootable image generation.
- Installer development.
- Installation and first-boot experience.
- System provisioning.
- Hardware compatibility testing.
- Distribution and release engineering.

Phase 5 should produce a functioning native GemmaOS environment that can become the foundation for a bootable distribution.

FINAL ARCHITECTURAL OBJECTIVE

GemmaOS transitions from a browser-hosted development environment into a native Linux desktop platform.

The original application architecture, AI infrastructure, semantic memory, and agent environment remain intact wherever possible.

The platform layer changes.

The operating environment changes.

The core system contracts remain stable.

The result is a native Linux implementation of GemmaOS, ready for bootable distribution engineering in Phase 6.

GEMMAOS

PHASE 6 — BOOTABLE OS & RELEASE

OBJECTIVE

Transform the native Linux implementation developed during Phase 5 into a bootable, installable, maintainable GemmaOS distribution.

Phase 6 establishes the complete operating-system delivery pipeline, including bootloader integration, system image generation, installation, first-boot configuration, hardware compatibility, release engineering, and distribution.

The objective is to deliver a functioning AI-native operating system that boots directly into the GemmaOS desktop environment.

GemmaOS must provide an integrated computing environment containing:

- Native desktop environment.
- Application framework.
- Gemma generative intelligence.
- EmbeddingGemma 2 semantic intelligence.
- AI orchestration infrastructure.
- Agent execution environment.
- Semantic memory and retrieval.
- Native Linux platform services.
- System security and resource management.

The release must be reproducible, testable, installable, and maintainable.

---

1. LINUX DISTRIBUTION FOUNDATION

Select the underlying Linux distribution and establish the GemmaOS system image architecture.

Distribution strategy

Build GemmaOS on an established Linux distribution rather than creating a kernel and Linux distribution ecosystem from scratch.

Evaluate candidates based on:

- Long-term support.
- Hardware compatibility.
- Package availability.
- Security updates.
- Desktop integration.
- Driver availability.
- System maintenance.
- Distribution licensing.
- Image customization capabilities.

Potential foundations include:

- Ubuntu.
- Debian.

- Fedora.

Select one primary distribution for the initial release.

Avoid maintaining multiple distribution targets before the first release is stable.

System composition

The GemmaOS distribution should include:

- Linux kernel supplied by the underlying distribution.
- Bootloader.
- Base userspace.
- Native system services.
- GemmaOS desktop shell.
- Application framework.
- AI infrastructure.
- Agent environment.
- System configuration.
- Installation and recovery utilities.

Acceptance criteria

A reproducible base operating-system image can be generated with the GemmaOS software stack integrated.

---

2. BOOTLOADER & BOOT PROCESS

Implement the boot process required to start GemmaOS.

Responsibilities

- Bootloader configuration.
- Kernel loading.
- Initial RAM filesystem configuration.
- Root filesystem discovery.
- System initialization.
- Service startup.
- Desktop startup.

Boot architecture

The expected boot sequence:

1. Firmware initializes the computer.
2. Firmware loads the configured bootloader.
3. Bootloader loads the Linux kernel.
4. Kernel initializes hardware and system infrastructure.
5. Initial userspace starts.
6. Native system services initialize.
7. GemmaOS dependencies are verified.
8. Display manager or configured startup environment launches GemmaOS.
9. GemmaOS desktop becomes available.

Bootloader considerations

Evaluate GRUB and systemd-boot according to the selected distribution and installation strategy.

Support UEFI systems as the initial target.

Document BIOS compatibility separately if required.

Acceptance criteria

A generated GemmaOS image boots successfully in a virtual machine.

---

3. ROOT FILESYSTEM & SYSTEM LAYOUT

Define the filesystem structure for the GemmaOS distribution.

Responsibilities

Establish a predictable separation between:

- Operating-system files.
- Applications.
- System configuration.
- User data.
- Model files.
- Semantic memory.
- Logs.
- Temporary runtime data.
- Recovery resources.

Example filesystem organization

`/etc/gemmaos/`
System configuration.

`/opt/gemmaos/`
GemmaOS application and system components where appropriate.

`/var/lib/gemmaos/`
Persistent system service data.

`/var/log/gemmaos/`
System and application logs.

`/home/`
User files and personal configuration.

`/usr/share/gemmaos/`
Shared application resources and assets.

Requirements

- Explicit data ownership.
- Appropriate filesystem permissions.
- Persistent user data.
- System configuration protection.
- Backup and recovery support.
- Storage migration compatibility.

Do not store all application and model data in one undifferentiated directory.

---

4. SYSTEM INSTALLER

Develop an installation workflow for GemmaOS.

Installer responsibilities

- Detect available storage devices.
- Display installation requirements.
- Identify existing operating systems.
- Configure installation targets.

- Create required partitions.
- Install the bootloader.
- Deploy the operating-system image.
- Configure system services.
- Create the initial user account.
- Configure networking.
- Finalize installation.

Installation modes

Evaluate:

- Virtual machine installation.
- Installation to a dedicated disk.
- Dual-boot installation.
- Live environment.

Start with virtual machine installation and dedicated-disk installation.

Dual-boot support should only be introduced after partitioning, bootloader, and recovery procedures have been thoroughly tested.

Safety requirements

- Clearly identify destructive disk operations.
- Require explicit confirmation before formatting.
- Validate installation targets.
- Provide installation failure recovery.
- Avoid modifying unrelated partitions.
- Preserve existing user data unless explicitly authorized.

Acceptance criteria

A user can install GemmaOS into a virtual machine using the documented installation workflow.

---

5. FIRST-BOOT EXPERIENCE

Build the initial system configuration experience.

First-boot responsibilities

- Welcome screen.
- User account setup.
- Network configuration.
- Time zone configuration.
- System preferences.
- Privacy settings.
- AI provider configuration.
- Model configuration.
- Optional memory initialization.
- Desktop initialization.

AI setup

Allow users to configure:

- Gemma runtime provider.
- EmbeddingGemma 2 runtime provider.
- Local inference availability.
- Remote inference credentials.
- Model preferences.
- Memory and retrieval settings.

Remote inference must be supported for computers without sufficient local hardware.

API credentials must be stored securely and must never be exposed through browser-facing application code.

Acceptance criteria

A newly installed system can complete initial configuration without requiring manual modification of internal configuration files.

---

6. AI-NATIVE SYSTEM INTEGRATION

Make AI infrastructure a first-class component of the operating system.

GemmaOS must not treat its AI capabilities as a disconnected application.

Generative intelligence

Gemma provides the generative model runtime.

Responsibilities include:

- Natural-language interaction.
- Generation.
- Reasoning and planning capabilities supported by the selected model.
- Structured output.
- Tool interaction.
- Streaming responses.

Semantic intelligence

EmbeddingGemma 2 provides semantic representation and embedding capabilities supported by the selected model configuration.

Responsibilities include:

- Semantic indexing.
- Document retrieval.
- Memory retrieval.
- Similarity search.
- Contextual search.
- Multimodal retrieval where supported.

AI orchestration

The AI orchestrator coordinates:

- Model selection.
- Inference requests.
- Semantic retrieval.
- Context assembly.
- Agent execution requests.
- Application integrations.

System-level AI capabilities

Expose authorized AI services to applications through stable system interfaces.

Potential capabilities include:

- Semantic file search.
- Contextual application search.
- AI-assisted system navigation.
- Document understanding.
- Natural-language interaction with authorized system tools.

- Personal knowledge retrieval.
- Workflow assistance.

Acceptance criteria

AI services initialize, operate, and recover as integrated system components.

Core desktop functionality must remain usable if an AI provider is unavailable.

---

7. AGENT OPERATING ENVIRONMENT

Integrate the Phase 3 agent infrastructure into the bootable operating system.

Required components

- Agent Runtime.
- Agent Orchestrator.
- Task Planner.
- Tool Registry.
- Permission Manager.
- Workflow Engine.
- Agent Memory Integration.
- Multi-Agent Coordination.
- Agent Activity Monitor.

System integration

Agents must be able to interact with authorized system capabilities through controlled tools.

Potential capabilities:

- Search authorized files.
- Retrieve semantic memories.
- Launch applications.
- Inspect system status.
- Manage approved workflows.
- Interact with configured AI services.
- Perform approved file operations.

Security boundaries

- Explicit tool permissions.
- User-level authorization.
- Human approval for consequential operations.
- Execution limits.
- Audit logging.
- Cancellation and recovery.
- No unrestricted root execution.

Acceptance criteria

The installed GemmaOS environment supports agent execution through the documented permission and orchestration architecture.

---

8. MODEL MANAGEMENT & RESOURCE PROVISIONING

Establish model installation and execution management.

Responsibilities

- Model configuration.
- Runtime provider selection.
- Model availability checks.
- Model download management where applicable.
- Model storage management.
- Model initialization.
- Model unloading.
- Runtime health monitoring.
- Resource-aware execution.

Hardware compatibility

Support multiple operating configurations:

1. Remote inference.
2. CPU-based local inference.
3. GPU-accelerated local inference.
4. Hybrid local and remote inference.

Important requirements

Do not require every installation to download large models immediately.

Model downloads should be optional and user-controlled.

Detect available hardware before recommending local inference.

Provide clear information about model storage and resource requirements.

Acceptance criteria

GemmaOS can configure compatible local or remote model providers without requiring a specific GPU configuration.

---

9. HARDWARE COMPATIBILITY

Validate the operating system across supported hardware configurations.

Initial targets

- x86-64 virtual machines.
- x86-64 desktop computers.
- x86-64 laptops.

Additional architectures, including ARM64, can be evaluated separately.

Hardware testing

Validate:

- CPU initialization.
- Memory detection.
- Storage devices.
- GPU availability.
- Display output.
- Keyboard and pointer input.
- Networking.
- Audio.
- Battery and power management where applicable.

Driver compatibility

Leverage the underlying Linux distribution’s kernel and driver ecosystem.

Avoid creating custom hardware drivers unless a concrete requirement justifies them.

Acceptance criteria

The release has a documented hardware compatibility matrix and passes testing on its declared target configurations.

---

10. SYSTEM UPDATES & RELEASE MANAGEMENT

Implement an operating-system update strategy.

Update responsibilities

- System package updates.
- GemmaOS application updates.
- AI subsystem updates.
- Security patches.
- Configuration migrations.
- Update verification.
- Rollback or recovery procedures.

Release engineering

Establish:

- Version numbering.
- Release channels.
- Build identifiers.
- Changelogs.
- Dependency manifests.
- Release artifacts.
- Integrity checksums.
- Software provenance.
- Reproducible build procedures where practical.

Update channels

Evaluate:

- Development.
- Testing.
- Stable.

Do not distribute unvalidated development builds through the stable channel.

Acceptance criteria

A GemmaOS installation can receive documented, verifiable updates without silently losing user data.

---

11. RECOVERY & SYSTEM RESILIENCE

Provide recovery mechanisms for failed installations, updates, and system startup.

Recovery capabilities

- Recovery boot option.
- System diagnostics.
- Startup failure reporting.
- Configuration recovery.
- User-data backup.
- System restoration.
- Failed-update recovery.
- Log inspection.

Recovery scenarios

1. Bootloader configuration becomes invalid.
2. System update fails.
3. Desktop startup fails.
4. AI services fail to initialize.
5. Model storage becomes unavailable.
6. Persistent storage becomes corrupted.
7. User configuration becomes invalid.

Acceptance criteria

A failed update or service startup must not automatically destroy user data or leave the installation unrecoverable.

12. DOCUMENTATION & USER EXPERIENCE

Prepare GemmaOS for users beyond its development environment.

Required documentation

- Installation guide.
- Hardware compatibility guide.
- First-boot guide.
- Desktop usage guide.
- AI configuration guide.
- Model management guide.
- Agent permissions guide.
- Troubleshooting guide.
- Recovery guide.
- Update guide.
- Security documentation.
- Release notes.

User experience

The operating system should provide a coherent identity.

Preserve the GemmaOS visual language:

- Lavender and violet.
- Saturated purple.
- Bubblegum and hot pink.
- Warm cream panels.
- Rounded toy-like interface elements.
- Deep plum outlines.
- Playful gradients.
- Cute icons and decorative details.

The visual identity must remain consistent across:

- Boot and login experience.
- Installer.
- Desktop.

- System settings.
- AI applications.
- Model manager.
- Agent manager.
- System notifications.

Visual polish must not compromise accessibility or usability.

---

13. DEVELOPMENT MILESTONES

P6.1 — Distribution Foundation

Select the Linux base and establish reproducible image generation.

P6.2 — Bootloader Integration

Configure the bootloader and validate kernel startup.

P6.3 — Root Filesystem Architecture

Establish the filesystem layout and data ownership model.

P6.4 — System Image Generation

Generate a bootable GemmaOS image.

P6.5 — Installer Development

Implement installation and partitioning workflows.

P6.6 — First-Boot Configuration

Implement user setup, networking, preferences, and AI configuration.

P6.7 — AI-Native Integration

Integrate Gemma, EmbeddingGemma 2, model management, and semantic memory.

P6.8 — Agent Environment Integration

Integrate agent execution, tools, permissions, and workflows.

P6.9 — Hardware Compatibility

Validate supported hardware configurations.

P6.10 — Updates and Recovery

Implement update management, diagnostics, and recovery mechanisms.

P6.11 — Release Engineering

Establish versioning, release artifacts, documentation, and distribution workflows.

P6.12 — Final Validation

Complete installation, boot, security, integration, performance, and recovery testing.

---

14. FINAL ACCEPTANCE CRITERIA

GemmaOS is ready for its initial release when:

- A bootable system image can be generated.
- The image boots successfully in a virtual machine.
- The installer can deploy GemmaOS.
- The system completes first-boot configuration.
- The native desktop launches automatically.
- Core system services initialize correctly.
- Applications launch and operate.
- Gemma generative intelligence is configurable.
- EmbeddingGemma 2 semantic intelligence is configurable.
- Semantic memory persists across restarts.
- Agent execution operates within enforced permissions.
- Hardware resources are detected.
- Local and remote inference options are supported where compatible.
- Updates can be applied safely.
- Recovery procedures are documented and tested.
- Security boundaries are validated.
- Release artifacts are versioned and documented.
- Installation and user documentation are complete.

---

15. RELEASE STRATEGY

Do not attempt to launch a fully mature general-purpose operating system immediately.

Use staged releases.

Alpha

Developer-focused bootable image.

Objectives:

- Validate boot.
- Validate installation.
- Validate native desktop.
- Validate core system services.

Beta

Feature-complete testing environment.

Objectives:

- Validate AI integration.
- Validate agent workflows.
- Validate persistence.
- Validate hardware compatibility.
- Identify stability problems.

Release Candidate

Production-readiness testing.

Objectives:

- Security review.
- Installation validation.
- Update validation.
- Recovery validation.
- Performance testing.
- Documentation completion.

Initial Stable Release

Publish the first supported GemmaOS distribution.

The initial release should prioritize reliability and a clearly defined hardware compatibility target over an unnecessarily large feature set.

FINAL ARCHITECTURAL OBJECTIVE

GemmaOS becomes an independently bootable and installable AI-native operating-system distribution.

It integrates:

- A native Linux desktop environment.
- Generative intelligence through Gemma.
- Semantic intelligence through EmbeddingGemma 2.
- Shared semantic memory infrastructure.
- AI orchestration.
- Agent execution and workflow management.
- Native system services.
- Security and permission enforcement.
- Persistent user data.
- Hardware-aware resource management.

The operating system must remain modular, maintainable, extensible, and capable of evolving independently of individual model providers.

The final result is not simply a Linux desktop with an AI chatbot installed.

It is an operating environment designed around integrated AI services, semantic retrieval, and authorized agent capabilities.